IMAGE PROCESSING

TUGAS 2

IMAGE ENHANCEMENT DAN HISTOGRAM EQUALIZATION

Nama: Siti Fadhilah Rahmi

NIM: 2311532003

Dosen Pengampu: Afdhal Dinilhak, S.Kom, M.Kom

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

A. Histogram Equalization

Histogram Equalization atau perataan histogram adalah mengubah derajat keabuan suatu pixel (r) dengan derajat keabuan yang baru (s) dengan suatu fungsi transformasi T, yang dalam hal ini s = T(r). Ini dilakukan untuk memperoleh penyebaran histogram yang merata, sedemikian sehingga setiap derajat keabuan memiliki jumlah pixel yang relatif sama. Rumus menghitung histogram:

$$P_r(r_k) = \frac{n_k}{n}$$

yang dalam hal ini,

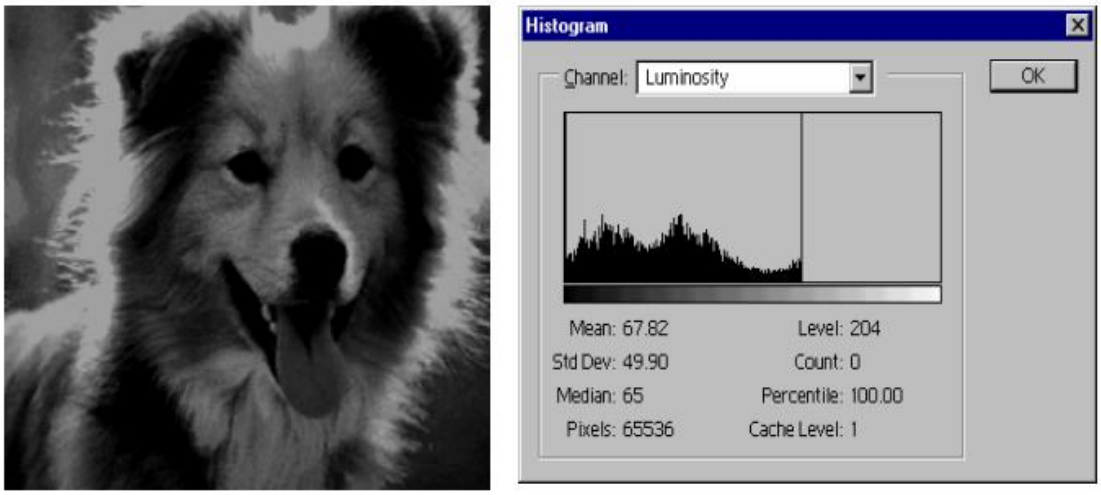
$$r_k = \frac{k}{L-1} \qquad , 0 \le k \le L-1$$

yang artinya, derajat keabuan (k) dinormalkan terhadap derajat keabuan terbesar (L – 1). Nilai rk = 0 menyatakan hitam, dan rk = 1 menyatakan putih dalam skala keabuan yang didefinisikan.

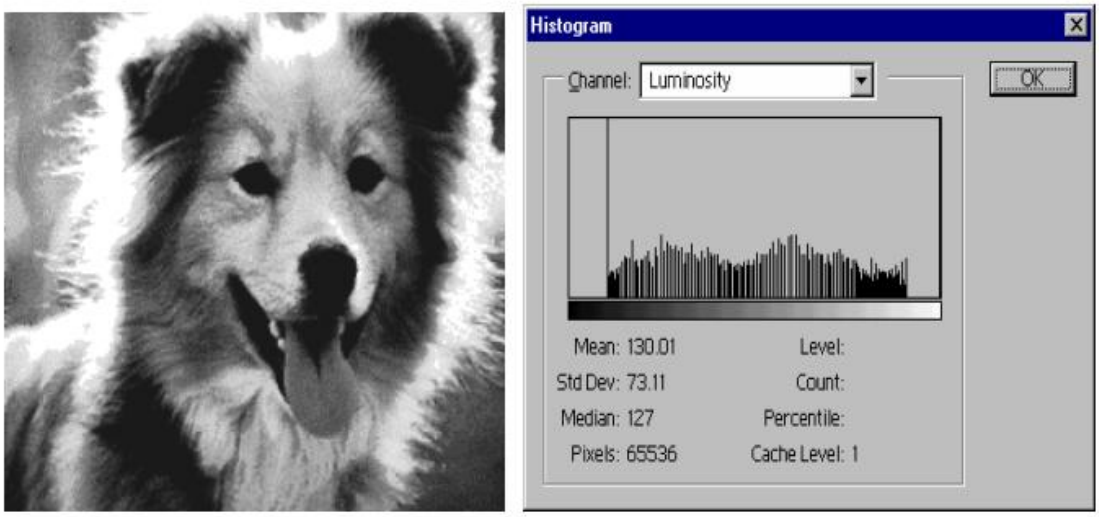
Meskipun perataan histogram bertujuan menyebarkan secara merata nilai-nilai derajat keabuan, tetapi seringkali histogram hasil perataan tidak benar-benar tersebar secara merata. Alasannya adalah

1. Derajat keabuan terbatas jumlahnya. Nilai intensitas baru hasil perataan merupakan pembulatan ke derajat keabuan terdekat.
2. Jumlah pixel yang digunakan sangat terbatas.

Agar hasil perataan benar-benar seragam sebarannya, maka citra yang diolah haruslah dalam bentuk malar (continue).



(a) Kiri: citra anjing *collie* yang terlalu gelap; Kanan: histogramnya



(b) Kiri: citra anjing *collie* setelah perataan histogram; kanan: histogramnya

B. Image Enhancement untuk Normalisasi Histogram dari Citra

Perbaikan kualitas citra (image enhancement) adalah proses mendapatkan citra yang lebih mudah diinterpretasikan oleh mata manusia. Pada proses ini, ciri-ciri tertentu yang terdapat di dalam citra lebih diperjelas kemunculannya.

Normalisasi histogram adalah teknik pengolahan citra yang mengubah histogram frekuensi intensitas piksel menjadi distribusi probabilitas, di mana nilai pada sumbu Y dinormalisasi ke rentang 0 – 1 dengan membagi jumlah piksel setiap intensitas dengan total seluruh piksel dalam citra. Proses ini bertujuan untuk menyamakan skala histogram, mempermudah perbandingan citra, dan meningkatkan kontras.

1. Normalisasi Histogram

   a. Sebelum membuat fungsi untuk normalisasi histogram, ada beberapa library yang digunakan, yaitu:

      - Matplotlib untuk visualisasi
      - Numpy untuk pengolahan angka, karena citra digital terdiri dari kumpulan angka dalam bentuk matrix
      - cv2 yang merupakan library opencv hanya digunakan untuk men-decode data mentah dari internet menjadi format citra (imdecode) dan mengubahnya menjadi grayscale.
      - Requests untuk melakukan HTTP request agar dapat mengambil gambar dari github

```
import matplotlib.pyplot as plt
import numpy as np
import cv2
import requests
```

   b. Ambil gambar dari github, lalu dirubah menjadi array Numpy dan gambar dirubah menjadi gambar hitam putih.

```
# Ambil Gambar dari GitHub
url = "https://raw.githubusercontent.com/SitiFadhilahRahmi/Image-Processing-Semester-6/e12708cb7a9053dfdeffd35accf5002976a492ee/image.jpeg"
resp = requests.get(url)
arr = np.frombuffer(resp.content, np.uint8)
img = cv2.imdecode(arr, cv2.IMREAD_GRAYSCALE)
```

   c. Membuat fungsi untuk menghitung histogram awal

      - Fungsi get_histogram_raw() menghitung frekuensi kemunculan setiap intensitas piksel (0-255)
      - Melakukan iterasi pada setiap piksel gambar (baris dan kolom) dengan nested loop
      - Setiap nilai intensitas yang ditemukan akan menambah nilai pada array index yang sesuai (Hist[image[i, j]] += 1)

```
# Fungsi Hitung Histogram (Sebelum Normalisasi)
def get_histogram_raw(image):
    N, M = image.shape
    Hist = np.zeros(256, dtype=int)
    for i in range(N):
        for j in range(M):
            Hist[image[i, j]] += 1
    return Hist
```

   d. Normalisasi Histogram

      - Fungsi get_histogram_normalized() menghitung total piksel dengan n = N * M

- Membagi setiap nilai histogram mentah dengan total piksel (hist_raw[i] / float(n))
- Hasilnya adalah probabilitas kemunculan setiap intensitas (nilai antara 0-1)

```
# Fungsi Normalisasi Histogram
def get_histogram_normalized(hist_raw, N, M):
    n = N * M
    hist_norm = np.zeros(256, dtype=float)
    for i in range(256):
        hist_norm[i] = hist_raw[i] / float(n)
    return hist_norm
```

e. Mengimplementasikan fungsi yang telah didefinisikan tadi ke dalam gambar

```
N, M = img.shape
# Sebelum Normalisasi
hist_raw = get_histogram_raw(img)

# Setelah Normalisasi
hist_norm = get_histogram_normalized(hist_raw, N, M)
```

f. Visualisasi Histogram

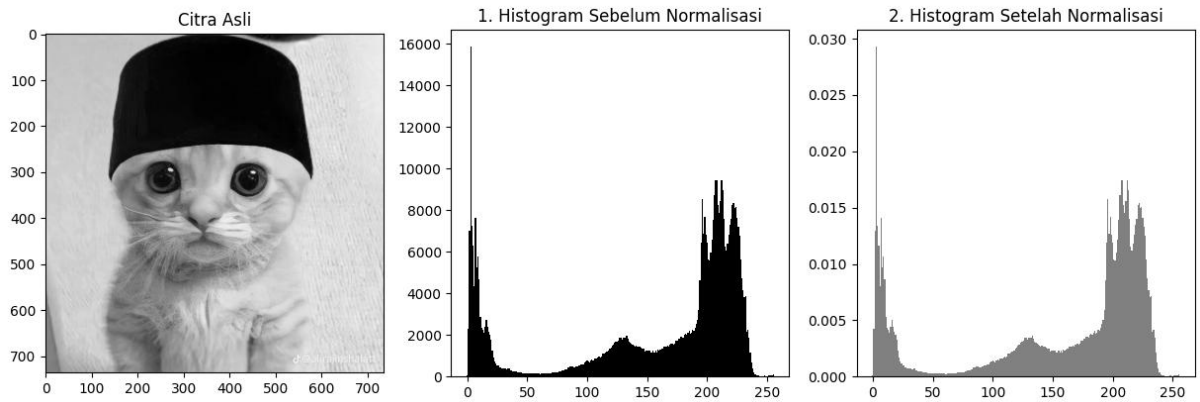
- Menampilkan citra asli
- Menampilkan histogram awal sebelum dinormalisasi
- Menampilkan histogram hasil normalisasi

```
# --- Visualisasi ---
plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1); plt.imshow(img, cmap='gray'); plt.title("Citra Asli")

plt.subplot(2, 3, 2)
plt.bar(range(256), hist_raw, color='black', width=1.0)
plt.title("1. Histogram Sebelum Normalisasi")

plt.subplot(2, 3, 3)
plt.bar(range(256), hist_norm, color='gray', width=1.0)
plt.title("2. Histogram Setelah Normalisasi")
```



Dari hasil visualisasi dapat dilihat bahwa bentuk histogram tidak berubah, namun rentang angka dari histogram yang berubah. Sebelum dinormalisasi, nilai pada

histogram berada di rentang 0 – 16.000, sementara setelah dinormalisasi, histogram berada di rentang 0 – 1.

2. Histogram Equalization

```python
# Fungsi Histogram Equalization
def histogram_equalization(image, hist_norm):
    cdf = np.zeros(256, dtype=float)
    cumulative_sum = 0
    for i in range(256):
        cumulative_sum += hist_norm[i]
        cdf[i] = cumulative_sum

    transform_map = np.floor(cdf * 255).astype(np.uint8)

    img_new = np.zeros_like(image)
    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            img_new[i, j] = transform_map[image[i, j]]
    return img_new

img_equalized = histogram_equalization(img, hist_norm)
hist_equalized_raw = get_histogram_raw(img_equalized)
hist_equalized_norm = get_histogram_normalized(hist_equalized_raw, N, M)
```

Pada tahap equalization ini, yang dilakukan adalah:

a. Menghitung CDF (Cumulative Distribution Function)
   - Fungsi histogram_equalization() menerima gambar dan histogram ternormalisasi
   - Menghitung kumulatif dari nilai-nilai histogram ternormalisasi dengan loop dari 0-255
   - Menyimpan hasil kumulatif ke dalam array cdf

b. Membuat Peta Transformasi
   - Mengalikan nilai CDF dengan 255 (nilai intensitas maksimum)
   - Menggunakan np.floor() untuk pembulatan ke bawah
   - Mengonversi ke tipe data uint8 (0-255) untuk menghasilkan peta transformasi

c. Menerapkan Transformasi pada Gambar
   - Membuat array kosong img_new dengan ukuran yang sama dengan gambar asli
   - Melakukan iterasi pada setiap piksel gambar asli
   - Mengganti nilai piksel lama dengan nilai baru dari transform_map berdasarkan intensitas aslinya

d. Menghitung Histogram Hasil Equalization
   - Fungsi get_histogram_raw() dipanggil untuk gambar hasil equalization
   - Fungsi get_histogram_normalized() menghitung histogram ternormalisasi dari gambar baru

e. Visualisasi hasil equalization

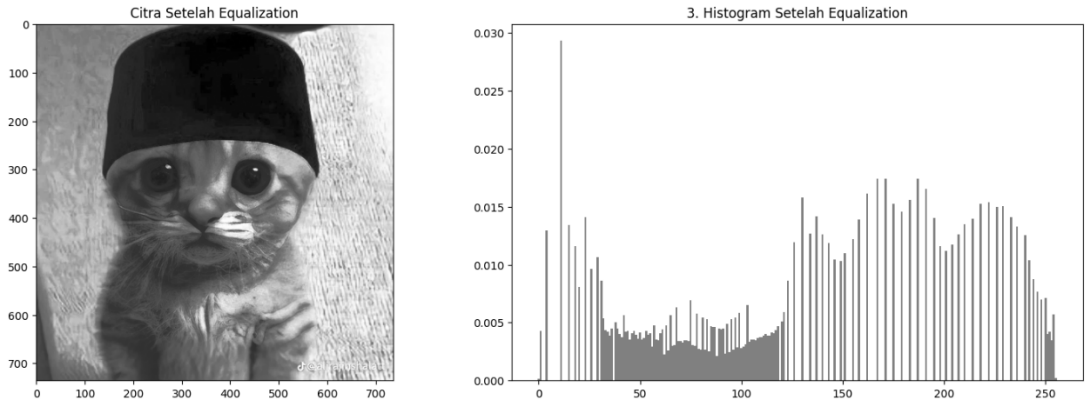
Visualisasi dilakukan dengan menampilkan gambar hasil equalization dan histogramnya

```
# --- Visualisasi ---
plt.figure(figsize=(15, 10))

plt.subplot(2, 2, 1); plt.imshow(img_equalized, cmap='gray'); plt.title("Citra Setelah Equalization")

# 3. Setelah Equalization
plt.subplot(2, 2, 2)
plt.bar(range(256), hist_equalized_norm, color='gray', width=1.0)
plt.title("3. Histogram Setelah Equalization")

plt.tight_layout()
plt.show()
```



Dari visualisasi di atas, dapat dilihat pada histogram, bahwa distribusi intensitas pixelnya menjadi lebih merata dan histogram terlihat lebih flat.

Kode lengkap dari program ini dapat diakses di github melalui link berikut: https://github.com/SitiFadhilahRahmi/Image-Processing-Semester-6/blob/main/Image_enhancement.ipynb